

Reviewer Pack — EF Refutation Size vs. Clause-Indicator Complexity

Entry ece004bc4f32 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-30 04:40:07 UTC

EF Refutation Size vs. Clause-Indicator Complexity

Entry ID: ece004bc4f32 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-30 04:40:07 UTC

1. Conjecture

Field A (mathematical branch): Extended Frege / Frege systems **Field B** (complexity object): Clause-indicator polynomials

Statement:

For any unsatisfiable CNF formula F , the size of the shortest Extended Frege refutation of F is polynomially related to the complexity of the clause-indicator polynomial of F , measured by the number of monomials and their degrees.

Rationale (proposer’s reasoning):

The complexity of the clause-indicator polynomial reflects the intricate structure of the CNF formula, and Extended Frege systems can efficiently encode and manipulate this structure. Thus, it is plausible that the size of the refutation is related to the complexity of the polynomial.

Taxonomy category: PROOF_COMPLEXITY_EXT_FREGE (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 5051cb6a1f401036

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	UNCERTAIN	SAFE
ALGEBRIZATION	UNCERTAIN	0.95	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
---------	---------	------------	------------------	------------------

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import subprocess
import json

def run_trial(seed: int) -> dict:
    random.seed(seed)

    # Generate a random unsatisfiable CNF formula
    n = 10 + random.randint(0, 20)
    m = 2 * n + random.randint(0, 4 * n)
    clauses = []
    for _ in range(m):
        clause = [random.choice([-1, 1]) * (i + 1) for i in range(n)]
        if len(set(clause)) == len(clause): # Ensure no duplicate literals
            clauses.append(clause)

    cnf_formula = "p cnf {} {} \n".format(n, m)
    cnf_formula += "\n".join(" ".join(str(l) for l in clause) + " 0" for clause in clauses)

    # Write the CNF formula to a temporary file
    with open("temp_cnf.cnf", "w") as f:
        f.write(cnf_formula)

    try:
        # Run minisat to find the refutation size
        result = subprocess.run(["minisat", "temp_cnf.cnf"], capture_output=True, text=True, timeout=90)
        if result.returncode != 0 and "UNSAT" in result.stderr:
            refutation_size = len(result.stderr.splitlines())
        else:
            return {
                "metric_name": "refutation_size",
                "metric_value": float('inf'),
                "instances_tested": 1,
                "conjecture_holds": False,
                "counterexample": "[Errno 2] No such file or directory: 'minisat'"
            }
    except subprocess.TimeoutExpired:
        return {
            "metric_name": "refutation_size",
            "metric_value": float('inf'),
            "instances_tested": 1,
```

```

        "conjecture_holds": False,
        "counterexample": "Timeout"
    }

    # Compute the complexity of the clause-indicator polynomial
    clause_indicator_poly = {}
    for clause in clauses:
        key = tuple(sorted(abs(l) for l in clause))
        if key not in clause_indicator_poly:
            clause_indicator_poly[key] = 0
        clause_indicator_poly[key] += 1

    poly_complexity = len(clause_indicator_poly)

    # Check if the refutation size is polynomially related to the polynomial complexity
    conjecture_holds = refutation_size <= poly_complexity ** 2

    return {
        "metric_name": "refutation_size",
        "metric_value": refutation_size,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if len(sys.argv) > 1 else [2**i + 7 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {json.dumps(result)}")
        results.append(result)

    refutation_sizes = [r["metric_value"] for r in results if "refutation_size" in r]
    poly_complexities = [r["instances_tested"] for r in results if "conjecture_holds" in r]

    mean_refutation_size = sum(refutation_sizes) / len(refutation_sizes)
    std_refutation_size = (sum((x - mean_refutation_size) ** 2 for x in refutation_sizes) / len(refutation_sizes)) ** 0.5
    support_fraction = sum(r["conjecture_holds"] for r in results) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_refutation_size} std={std_refutation_size} support_f={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results) and len(results) >= 30:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_db1422fa.py", line 87, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_db1422fa.py", line 40, in run_trial
    result = subprocess.run(["minisat", "temp_cnf.cnf"], capture_output=True, text=True, timeout=8)
              ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.12/subprocess.py", line 548, in run
    with Popen(*popenargs, **kwargs) as process:
         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.12/subprocess.py", line 1026, in __init__
    self._execute_child(args, executable, preexec_fn, close_fds,
  File "/usr/lib/python3.12/subprocess.py", line 1955, in _execute_child
    raise child_exception_type(errno_num, err_msg, err_filename)
FileNotFoundError: [Errno 2] No such file or directory: 'minisat'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to missing Minisat executable, preventing data collection | next: Install Minisat and retry tests with valid SAT solver configuration

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	groq	llama-3.3-70b-versat	0	0	21979
2	preregistration	ollama_remote	qwen3:8b	0	0	24244
3	novelty	ollama_remote	qwen3:8b	0	0	20758
4	novelty	ollama_remote	qwen3:8b	0	0	14307
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21200
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11066
7	judge	ollama_remote	qwen3:8b	0	0	15463

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 129017 ms total latency. Provider mix: {'groq': 1, 'ollama_remote': 6}

(full prompt+response transcripts available in research/audit/ece004bc4f32.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/ece004bc4f32.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/ece004bc4f32.tar.gz` (if generated)